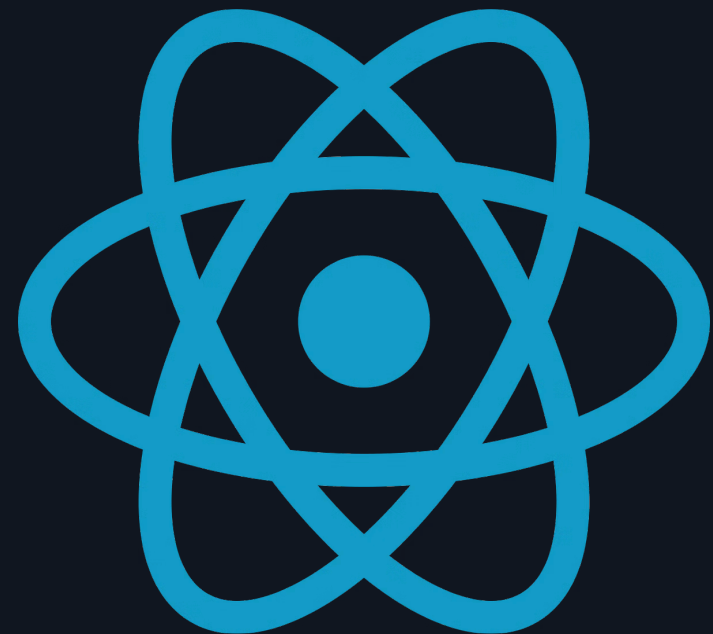




React

Librería JavaScript de código abierto para construir interfaces de usuario mediante componentes interactivos y reutilizables.

ANTONIO CASADO VICO

OLIVER MATTHEW RUTTER FRANCO

RAFAEL OSUNA RAMÍREZ

CRISTINA RAMOS MORA

SERGIO HERNÁNDEZ VAQUERO

ANTONIO JOSÉ HEREDIA FERNÁNDEZ

B → CONOCIMIENTOS Y HERRAMIENTAS

¿Qué necesitas para empezar?



Conocimientos previos

HTML, CSS y JavaScript son fundamentales. Git es recomendable.



Lenguajes

Obligatorios: **JavaScript** y **JSX**.
Recomendable: **TypeScript**.



Herramientas

Next.js (framework), **React Developer Tools** (inspección de componentes) y **Node.js** (dependencias y compiladores locales).

C → VENTAJAS

Ventajas de usar React

Componentes reutilizables

Divide la interfaz en bloques independientes, ahorrando tiempo y facilitando el mantenimiento.

Flujo unidireccional

Los datos viajan en una sola dirección, haciendo el código más predecible y fácil de depurar.

DOM Virtual

Solo se actualizan las partes necesarias del DOM real, mejorando el rendimiento.



aivs5lcchs



Pul' Sacres



Made with GAMMA

Inconvenientes de usar React



Sin estructura oficial

Cada equipo decide cómo organizar el código, lo que puede derivar en proyectos desordenados.

Curva de aprendizaje

Mezclar HTML con lógica JS mediante JSX requiere un dominio sólido del lenguaje.

Ecosistema cambiante

Las herramientas evolucionan rápido, exigiendo actualización constante.

- ❑ React es una librería minimalista, no un framework completo: obliga a depender de herramientas externas.

E → USO RECOMENDADO

¿Cuándo usar React?

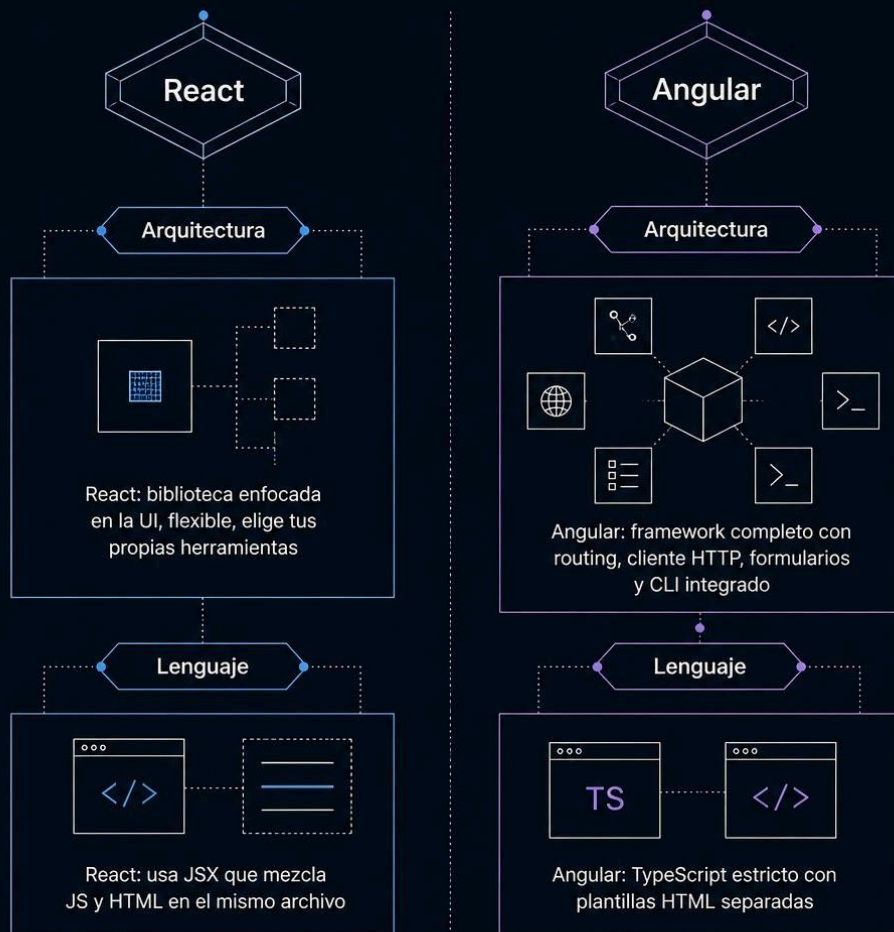
✓ Elegir React

Aplicaciones con **alta interactividad y estado complejo**: dashboards, filtros dinámicos, actualizaciones en tiempo real (paneles de administración, redes sociales). Flexibilidad para elegir router y gestor de estado.

✗ No elegir React

Webs sencillas sin interactividad (página de negocio local). React complicaría innecesariamente el proyecto y ralentizaría la carga. HTML/CSS o plantillas estáticas bastan.

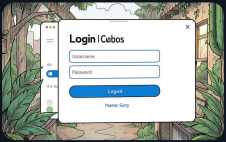
React vs Angular



¿Por qué React?

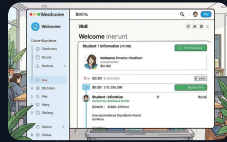
Ofrece mayor **flexibilidad** para organizar la aplicación sin arquitectura cerrada. Permite reutilizar conocimientos de JavaScript y aprender de forma progresiva. Ideal para proyectos como el nuestro: validar login, mostrar perfiles y filtrar asignaturas.

Componentes de la aplicación



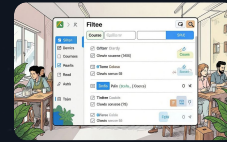
Login & Formulario

Inicio de sesión con validación de campos y errores de formato. Incluye acceso a inscripción.



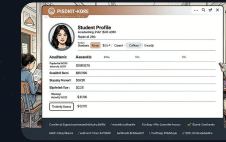
Home & HomeHero

Contenedor principal y área de bienvenida. Mantiene estado compartido: usuario, filtros y asignaturas.



SeleccionCursos & FiltrosBusqueda

Lista de asignaturas con barra de búsqueda y filtros dinámicos que se comunican entre componentes.



PerfilAlumno & Footer

Información del alumno y pie de página con enlaces, avisos y datos de contacto.

Responsabilidades: navegador vs servidor

Navegador (cliente)

- Mostrar interfaz y detectar interacciones
- Validaciones básicas: campos obligatorios, formato email
- Actualizar búsqueda y filtros en tiempo real
- Mostrar estados de carga, vacíos y errores

Servidor

- Verificar credenciales y gestionar sesiones
- Validar todos los datos del formulario
- Comprobar autorización en cada petición
- Devolver solo los datos permitidos al usuario



Las validaciones del cliente pueden evitarse. La seguridad real siempre está en el servidor.

Seguridad y Web-Grafía

Riesgos a gestionar

- No confiar solo en validaciones de React (ejecutadas en navegador)
- Almacenar contraseñas de forma segura
- Autorizar acceso al expediente en servidor
- Mantener dependencias actualizadas

Fuentes principales

- es.react.dev
- [The React Foundation \(2026\)](#)
- [Linux Foundation](#)
- [MDN Web Docs](#)
- [EBAC](#), [Qindel](#), [SoyHenry](#), [OpenWebinars](#), [Pontia](#)